

技术混合受众 · 全部静态回放

Vibe Coding

从需求到证据的受控开发闭环

一次真实工作过程，逐页拆开。

90 分钟

34 页

不跟做

无考核

学完能做什么

01

判断
AI 该做多少

02

开一次
受控会话

03

用四层证据
确认做对了

现场不调用 Agent，全部内容为静态回放。

模型 / Agent / tool / session

模型	推理引擎
Agent	执行主体
tool	可调用能力
session	一次工作容器

SESSION 边界

工作目录
上下文
权限

会话是本课的操作单元

受控开发

人定义四件事

01

目标

02

约束

03

决策

04

验收证据

AI 辅助
分析与实现

不是
“凭感觉接受输出”

安全红线

01

不上传敏感信息

02

不展示密钥

03

AI 无生产权限

04

高风险操作人工确认

不确定，就停下确认。

开始前：仓库与 Git

repository — clean starting point

```
$ git status --short
```

```
# no output
```

```
$ git branch --show-current
```

```
training/vibe-coding-reminder-window
```

开始前只确认两件事

仓库干净

分支明确

干净、可回退的起点
才值得信任。

OpenCode 是什么

一次可检查的编码会话

模型 + 工具 + 权限

价值不在打字快，
而在每一步都能检查。

OpenCode · opencode.ai

对话

文件差异

终端

首次会话·步骤 1：启动与状态

先看状态，再看任务

session status — static replay

```
model    selected / visible
workspace /project/repository
tools    read · search · terminal
permissions allow / ask by rule
```

状态回答

我在哪？
能看什么？
能做什么？

状态是会话的
边界说明书。

首次会话·步骤 2：只读理解

先验证理解，再谈修改

只读请求 · 静态回放

读取仓库，回答：
项目是什么？如何组织？如何构建？

限制：只读，不修改任何文件。

理解错了，
后面一切免谈。

回答先与仓库事实核对。

首次会话·步骤 3：模式与权限

模式与权限：allow / ask / deny

session / agent + permission

本回放显式切到 Plan

- 默认主 Agent：Build
- 默认多数 permission：allow
- Plan 默认编辑 / bash：ask

PERMISSION REQUEST

这次操作命中 ask

看清请求，再批准

allow 直接放行

ask 请求批准 · deny 直接拒绝

首次会话·步骤 4: diff / undo / 验证

改完以后：把关、回退、验证

DIFF	/UNDO · /REDO	VERIFY
范围有没有夹带？	撤销上一条用户消息	tests
+ intended change	含后续回复与改动	typecheck
– unrelated edit	可连续撤销；redo 恢复	build / E2E

diff 把关 · /undo 可连续回退 · /redo 恢复 · 跨会话 / 版本点用 Git

工具选择地图

工具不是一张品牌清单，
而是四种不同职责。

skills

固化 workflow

编排插件

复杂任务分角色

OpenSpec

组织规范与证据

MCP: 本项目无配置
本课不演示

MCP

只处理外部连接边界

skills: 标准 vs 技能包

开放标准 / 生态

Agent Skills

skills.sh

定义如何写与分发

标准定义形式；技能包提供具体内容。

具体技能包

mattpocock/skills

个人维护

不是官方标准

mattpocock/skills 是什么、为什么

个人技能集合 · 可复用 workflow

skills/grill-with-docs/SKILL.md

先读文档，再追问

- 把模糊词变成可验证问题
- 把确认结果写回约定

grill-with-docs

把歧义问成约束

diagnosing-bugs

先证据后改码

tdd

先红后绿

code-review

检查实现质量

四个技能覆盖受控开发的四个环节。

安装与审阅

示例命令 · 课堂不执行

```
$ npx skills@latest add mattpocock/skills
```

review / installed files

启用前审阅

- 看清写入了哪些文件
- 确认内容、setup 与权限边界

三条纪律

安装时选择

`setup-matt-pocock-skills`

每仓库运行一次

`/setup-matt-pocock-skills`

插件安装与 `skills.sh` 二选一

版本以官网 / 上游为准

grill / tdd / diagnose / code-review 用法

grill	先文档，后提问	01
tdd	先写失败测试	02
diagnose	先收集证据，再改码	03
code-review	看安全、清晰、可维护	04

四句用法，就是后半场演示的骨架。

何时单 Agent

一个 Agent 就够

边界清晰

串行依赖

一个人能描述清楚

考虑编排

能拆成独立子问题

需要并行

需要独立 / 对抗复核

能单 Agent，就不编排。

oh-my-opencode-slim 是什么、为什么

第三方社区插件 · 非 OpenCode 官方

长任务会带来
上下文膨胀与注意力漂移

复杂任务分职责；
简单任务不要用。

Orchestrator

Oracle

Librarian

Fixer

Explorer

Council

Designer

Observer · 可选

“计划 → 实现 → 复核”是抽象阶段，不是角色名。

安装 / 配置 / 权限

shell · 课堂不执行

```
$ npx oh-my-opencode-slim@latest install  
→ 完成登录 / 刷新  
  
$ opencode
```

配置路径 · 安装不创建 agent 目录

用户级 ~/.config/opencode/oh-my-opencode-slim.json(c)

项目级 .opencode/oh-my-opencode-slim.json · 可选覆盖

环境变量 · 后台编排 / 整体禁用

OPENCODE_EXPERIMENTAL_BACKGROUND_SUBAGENTS=true

读写配置 / 调用模型需看清权限 · Council 多模型投票成本高

OpenCode 对话

> ping all agents

OH_MY_OPENCODE_SLIM_DISABLE=1

agents 分工与最小用法

01

Orchestrator

分派与汇总

04

Oracle

按需高风险评审

02

Explorer

只读侦察

05

Council

多模型投票 · 成本高

03

Fixer

有界实现

Designer: UI
Librarian: 官方资料

按需启用；本课演示主要使用单 Agent。

OpenSpec 是什么、为什么

把“符合需求”
变成可核对的证据。



OpenSpec · openspec.dev

目录与 artifacts

openspec / directory

openspec/

- ├─ changes/<change>/
 - ├─ proposal.md
 - ├─ specs/.../spec.md
 - ├─ design.md
 - └─ tasks.md
- └─ changes/archive/...

proposal

为什么改

spec

SHALL + 场景

design

设计决策

tasks

最小步骤

培训夹具位于 docs 下，
不进入 CLI 扫描。

CLI vs 聊天命令

CLI · 给人操作

终端中直接运行

适合手动检查

由人控制参数

聊天命令 · 给会话使用

本仓库使用连字符

Agent 在会话内调用

项目已有资产

```
/opsx-propose /opsx-apply /opsx-verify /opsx-archive /opsx-explore
```

propose → apply → verify → archive

explore

摸现状

propose

出规格

apply

按规格实现

archive

完成后归档

`.opencode/commands/opsx-verify.md`

项目已有启发式命令

- 读取 artifacts · 搜索代码
- 判断完整性 / 正确性 / 连贯性
- 无需 profile · 不自动运行测试

STRICT

格式 · requirement / scenario
结构与可解析性

不证明 artifacts 完成度或行为

MCP 边界

内置工具不够，
又必须连接外部系统时，才考虑 MCP。

它是协议，不是需要安装的单一运行时。

本项目边界

无 MCP 配置
本课不演示

别为用而用。

TDD vs diagnosis

TDD · 已知目标行为

“保存后应立即刷新”

第一步
写一个会失败的测试

项目案例完整演示

DIAGNOSIS · 未知故障原因

虚构教学微案例
搜索框 Enter 失败 · 点击正常

- | | |
|-------------|---|
| 01 复现 / 最小化 | 只留搜索表单与 keydown 路径 |
| 02 假设 | 焦点 · preventDefault · submit 绑定 |
| 03 插桩 / 排除 | keydown 已触发 · submit handler=0
排除焦点 / 后端 |
| 04 修复 / 回归 | 统一 form onSubmit
Enter / 点击各触发 1 次 |

独立于项目四阶段 · 不声称来自真实缺陷

四层证据

01	strict	规格结构完整	不证明行为
02	规范映射 / 启发式核对	历史人工/等效规范映射 /opsx-verify 可辅助	当时未运行命令；不覆盖场景外
03	机器执行证据	tests：被断言行为 · typecheck：类型一致 build / package：可构建打包 · E2E：特定环境路径	不证明未断言行为 / 其他环境 / 代码质量
04	code review	安全、清晰、可维护	不穷尽所有行为

四层互补；机器结果不能读成“全部正确”。

演示原始需求

原始需求

“在提醒面板里让负责人配置临期窗口。”

一句话里，藏着五个可验证的歧义。

业务背景

临期
今日到期
已逾期

来自项目 · 范围小
可观察 · 已脱敏

grill 澄清

Q1	0 合法吗?	合法；未来提醒不标临期
Q2	能填多大?	0..9007199254740991
Q3	何时生效?	显式保存后刷新并持久化

保存前不生效 · 保存后立即刷新 · 重开保持 · 提醒日期可见

专用 training change / spec

docs/training/vibe-coding/training-change/spec.md

Scenario: 临期窗口为 0

- GIVEN 配置为 0 并显式保存
- THEN 未来提醒不归入临期
- AND 今日到期 / 已逾期不受影响

归档漂移 · 三条事实

原 change 已归档

正式 spec 仍是旧版

最终实现未合入 main

openspec/changes/archive/
2026-08-10-add-relocation-
service-workbench/

培训夹具，无产品批准语义

Red 与 preset-failure

RED-TEST · f771667

historical output

domain + integration 32 passed

main 2 failed

V2_MUTATION_UNKNOWN

renderer 1 failed

未找到“临期窗口”控件

PRESET-FAILURE · 0554164

historical output

IPC / facade / UI connected

0 case failed

“必须不小于 1”

negative / fraction rejected

renderer: mutation not called

红：行为已经写明，实现还没跟上。表面能点，也可能做错。

Green / review / verify

green-final · d304bd5 · historical

focused 95 passed

full 1076 passed

strict valid (structure)

e2e build passed

layout E2E 1 / 1

review / historical ledger

质量门

- 2 个 medium 已修正；2 个既有 low 保留
- 最后三项修正后未做第三轮独立 Oracle 评审
- 当前是课程内容终审

UI: 填 0 → 保存 → 立即刷新 → 日期可见
今日到期 / 已逾期不变

strict: 2026-08-10 历史摘要
规范映射: 历史人工 / 等效核对
当时未运行 / opsx-verify

日常落地 / 风险适配

01

先文档后提问

02

写清约定与证据

03

TDD 先红

04

diff 把关再绿

05

review 看质量

06

verify 对约定

低

简短约定 + 聚焦验证

中

加深文档 / 测试 / review

高

人工确认 + 完整证据

目标、边界、证据
任何强度都不能省。

跨 shared IPC / main / renderer：集成风险按中等

先答共性，再记个性。

个人配置与扩展问题进入待答问题池，会后书面处理。

受控

证据

闭环

明天开始：先写目标、边界和验收证据，再打开会话。